### 7.4.4 Restricted file downloads

So far, all the work with uploaded files has resulted in files that can be downloaded without any restrictions. Everything at /media/ (or wherever your `MEDIA_URL` points) is available to all users, even those that aren't logged in. If you need to restrict downloads to logged-in users or wish to write complex logic for which users can see which files, you have two choices: a custom view or a hand-off view.

In either case, the uploaded file does not get stored in `MEDIA_ROOT` but somewhere else. Only Django and the web server have access to these kinds of files, and they aren't served under /media/, giving you control over who can access them.

The custom view choice is quick, dirty, and goes against recommended best practices. `FileResponse`, in `django.http`, works similar to `HttpReponse`, but it allows you to return a binary file object. It has both a streaming mode to return files inline (90s-style custom-GIF visitor counters anyone?) and an attachment mode to prompt the user for a download.

Django isn't optimized for dealing with binary files, and the quick-and-dirty choice doesn't handle caching or other optimization goodness unless you code it yourself. Even with fine-tuning, you're unlikely to be able to beat the performance of a web server.

The second choice is to take advantage of web server features. The two most popular web servers, Apache and Nginx, both support HTTP headers allowing you to hand off a download request. In this case, you still write a view, but the view returns HTTP headers that point to the file. With the right header in place, the web server does the rest of the work for you. The challenge with this approach is the headers are nonstandard and are different on different web servers. If you want to search the web for examples of how to do this, the header for Apache is `X-Sendfile` and the header for Nginx is `X-Accel-Redirect`.

Alternatively, you can skip all the hard work and use a third-party library. Both `django-private-storage` and `django-sendfile2` provide tools for serving uploaded files from a restricted folder.

## 7.5 Exercises

1. Modify the `musician()` view to display an optional description and bio-pic. Add an `edit_musician()` view, so users can self-serve the creation and editing of their musician profiles. Insert an Add Musician Profile link to the home page and an Edit link to the musician details page if the logged-in user is the owner. Allow staff members to edit `Musician` objects as well.
2. Modify the `seeking_ad()` view so that staff and administrators are allowed to edit an ad. Don't forget to change list_ads.html, so the edit links show up as well.

## Summary

- A Django `Form` class helps you process HTML `<form>` submissions, using form field attributes to describe each input.